```
#from google.colab import drive
#drive.mount('/content/drive')
```

```
#!git clone https://github.com/sunithak999/AAI-590-Capstone-group2.git
```

```
#!git clone https://github.com/TEJSINGH17/AAI-590-Capstone-group2 json
```

```
!pip install ultralytics opencv-python-headless
```

```
!pip -q install imageio imageio-ffmpeg
```

```
# ============================================================
# OmniView AI HUD Demo
# Pipeline:
#   1) Reads an MP4 video + per-frame JSON detection files
#   2) Overlays bounding boxes, blind-spot alerts,
#      distance tags, and a top status bar
#   3) Saves annotated frames as a new MP4
#   4) Converts the first N frames to an animated GIF
#   5) Displays the GIF inline in Google Colab
# ============================================================
```

```
# ============================================================
# Imports
# ============================================================

import cv2
import json
import os
from glob import glob
import imageio.v2 as imageio
from IPython.display import Image, display
```

```
# ============================================================
# PATHS
# ============================================================
#VIDEO_PATH = "/content/AAI-590-Capstone-group2/test_data/output_ds_1.mp4"
VIDEO_PATH = "/home/logicpro09/omniview_ai/output_ds_3_reenc.mp4"

# One JSON file per frame, written by the YOLO detector
#JSON_DIR = "/content/json/runs/hud/ds_1"
JSON_DIR = "/home/logicpro09/omniview_ai/runs/hud/ds_3"

#OUTPUT_VIDEO_PATH = "/content/omniview_hud_demo.mp4"
OUTPUT_VIDEO_PATH = "/home/logicpro09/omniview_hud_demo.avi"
#OUTPUT_GIF_PATH   = "/content/omniview_hud_demo.gif"
OUTPUT_GIF_PATH = "/home/logicpro09/omniview_hud_demo.gif"

# ============================================================
# SETTINGS
# ============================================================
MAX_FRAMES  = 60    # Cap at 60 frames to keep runtime manageable
CONF_THRESH = 0.40  # Drop anything below 40% confidence
GIF_FRAMES  = 40    # How many frames go into the GIF
GIF_FPS     = 10
```

```
# ============================================================
# OPEN VIDEO
# ============================================================
cap = cv2.VideoCapture(VIDEO_PATH)
if not cap.isOpened():
    raise ValueError(f"Could not open video: {VIDEO_PATH}")

# Fall back to 30 fps if the container reports 0
fps = cap.get(cv2.CAP_PROP_FPS)
if fps <= 0:
    fps = 30.0

width  = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
```

```
print(f"Input video: {VIDEO_PATH}")
print(f"Resolution: {width} x {height}")
print(f"FPS: {fps:.2f}")

# ============================================================
# GET JSON FILES
# ============================================================
# Sorted so files are read in frame order
json_files = sorted(glob(os.path.join(JSON_DIR, "*.json")))[:MAX_FRAMES]
if not json_files:
    raise ValueError(f"No JSON files found in: {JSON_DIR}")

print(f"JSON files used: {len(json_files)}")
print(f"Approx processed clip duration: {len(json_files)/fps:.2f} seconds")
```

```
# ============================================================
# OUTPUT VIDEO WRITER
# ============================================================
#fourcc = cv2.VideoWriter_fourcc(*"mp4v")
fourcc = cv2.VideoWriter_fourcc(*'XVID')
out = cv2.VideoWriter(OUTPUT_VIDEO_PATH, fourcc, fps, (width, height))
if not out.isOpened():
    raise ValueError("Could not open video writer. Check codec/path.")

# ============================================================
# HUD CONFIG
# ============================================================
vehicle_classes = {"car", "truck", "bus", "motorcycle", "bicycle"}

# Blind-spot zones cover the lower ~52% of the frame, 22% width on each side
left_zone  = (0,               int(height * 0.48), int(width * 0.22), height)
right_zone = (int(width * 0.78), int(height * 0.48), width,           height)


def draw_transparent_box(frame, pt1, pt2, color, alpha=0.18, thickness=2):
    """
    Semi-transparent filled rectangle with a solid border on top.
    alpha controls fill opacity -- 0.18 keeps it visible without blocking the scene.
    """
```

```
    overlay = frame.copy()
    cv2.rectangle(overlay, pt1, pt2, color, -1)
    cv2.addWeighted(overlay, alpha, frame, 1 - alpha, 0, frame)
    cv2.rectangle(frame, pt1, pt2, color, thickness)


def point_in_zone(cx, cy, zone):
    """Check if a bounding box center falls inside a zone rectangle."""
    x1, y1, x2, y2 = zone
    return x1 <= cx <= x2 and y1 <= cy <= y2


def draw_label(frame, text, x, y, color):
    """
    Text label with a filled background so it stays readable over any scene.
    Clamps y so the label never clips off the top of the frame.
    """
    (tw, th), _ = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, 0.55, 2)
    y = max(th + 8, y)
    cv2.rectangle(frame, (x, y - th - 8), (x + tw + 8, y), color, -1)
    cv2.putText(
        frame, text, (x + 4, y - 4),
        cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 2
    )


def estimate_distance(box_height_px):
    """
    Rough distance estimate based on bounding box height.
    Thresholds tuned for a typical dashcam FOV -- not exact, but good enough
    for a visual warning system.
    """
    if box_height_px >= 140:
        return "CLOSE"
    elif box_height_px >= 70:
        return "MEDIUM"
    else:
        return "FAR"


def safe_box_from_normalized(det, frame_w, frame_h):
```

```
    """
    Convert YOLO normalized (x_center, y_center, w, h) to pixel corner coords.
    Clamps to frame bounds so nothing draws outside the image.
    """
    xc = det["x_center"] * frame_w
    yc = det["y_center"] * frame_h
    bw = det["width"]    * frame_w
    bh = det["height"]   * frame_h

    x1 = int(xc - bw / 2)
    y1 = int(yc - bh / 2)
    x2 = int(xc + bw / 2)
    y2 = int(yc + bh / 2)

    x1 = max(0, min(frame_w - 1, x1))
    y1 = max(0, min(frame_h - 1, y1))
    x2 = max(0, min(frame_w - 1, x2))
    y2 = max(0, min(frame_h - 1, y2))

    return x1, y1, x2, y2
```

```
# ============================================================
# PROCESS VIDEO + JSON
# ============================================================
processed_frames = 0

for idx, json_path in enumerate(json_files):
    ret, frame = cap.read()
    if not ret:
        print(f"Video ended early at frame index {idx}")
        break

    with open(json_path, "r") as f:
        data = json.load(f)

    detections = data.get("detections", [])

    # Top status bar
    cv2.rectangle(frame, (0, 0), (width, 60), (20, 20, 20), -1)
```

```
        cv2.putText(
            frame, "OmniView AI | HUD Prototype", (20, 38),
            cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 255, 255), 2
        )

        # Blind-spot zone overlays
        draw_transparent_box(frame,
                             (left_zone[0],  left_zone[1]),
                             (left_zone[2],  left_zone[3]),
                             (0, 255, 255))
        draw_transparent_box(frame,
                             (right_zone[0], right_zone[1]),
                             (right_zone[2], right_zone[3]),
                             (0, 255, 255))

        cv2.putText(
            frame, "LEFT BLIND SPOT", (10, left_zone[1] - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 2
        )
        cv2.putText(
            frame, "RIGHT BLIND SPOT", (width - 210, right_zone[1] - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 2
        )

        # Reset alerts each frame
        left_alert  = False
        right_alert = False

        for det in detections:
            conf = float(det.get("confidence", 0.0))
            if conf < CONF_THRESH:
                continue

            label = str(det.get("label", "object"))

            try:
                x1, y1, x2, y2 = safe_box_from_normalized(det, width, height)
            except KeyError:
                continue

            if x2 <= x1 or y2 <= y1:
```

```
            continue

        cx = (x1 + x2) // 2
        cy = (y1 + y2) // 2
        box_height   = y2 - y1
        distance_tag = estimate_distance(box_height)

        color = (0, 255, 0)  # Default green; flips red if in a blind spot

        if label.lower() in vehicle_classes:
            if point_in_zone(cx, cy, left_zone):
                color = (0, 0, 255)
                left_alert = True
            elif point_in_zone(cx, cy, right_zone):
                color = (0, 0, 255)
                right_alert = True

        cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
        cv2.circle(frame, (cx, cy), 4, color, -1)

        label_text = f"{label} {conf:.2f} | {distance_tag}"
        label_y = y1 - 8 if y1 > 24 else y1 + 22
        draw_label(frame, label_text, x1, label_y, color)

    # Alert banners
    if left_alert:
        cv2.rectangle(frame, (20, 75), (420, 115), (0, 0, 255), -1)
        cv2.putText(
            frame, "ALERT: VEHICLE LEFT", (35, 103),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2
        )

    if right_alert:
        cv2.rectangle(frame, (20, 125), (430, 165), (0, 0, 255), -1)
        cv2.putText(
            frame, "ALERT: VEHICLE RIGHT", (35, 153),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2
        )

    out.write(frame)
    processed_frames += 1
```

```
cap.release()
out.release()

print(f"Saved MP4: {OUTPUT_VIDEO_PATH}")
print(f"Frames written: {processed_frames}")
print(f"MP4 exists: {os.path.exists(OUTPUT_VIDEO_PATH)}")
if os.path.exists(OUTPUT_VIDEO_PATH):
    print(f"MP4 size: {os.path.getsize(OUTPUT_VIDEO_PATH)} bytes")
```

```
# ============================================================
# CONVERT OUTPUT VIDEO TO GIF
# ============================================================
# imageio handles palette-optimized GIFs
gif_source_frames = []
reader = imageio.get_reader(OUTPUT_VIDEO_PATH)

for i, frame in enumerate(reader):
    if i >= GIF_FRAMES:
        break
    gif_source_frames.append(frame)  # imageio returns RGB

reader.close()

if not gif_source_frames:
    raise ValueError("No frames were read from the output video for GIF conversion.")

imageio.mimsave(OUTPUT_GIF_PATH, gif_source_frames, fps=GIF_FPS)

print(f"Saved GIF: {OUTPUT_GIF_PATH}")
print(f"GIF exists: {os.path.exists(OUTPUT_GIF_PATH)}")
if os.path.exists(OUTPUT_GIF_PATH):
    print(f"GIF size: {os.path.getsize(OUTPUT_GIF_PATH)} bytes")
    print(f"GIF duration: {len(gif_source_frames)/GIF_FPS:.2f} seconds")
```

```
# ============================================================
# DISPLAY GIF IN COLAB - uncomment when running
```

```
# ==========================================================
display(Image(filename=OUTPUT_GIF_PATH))
```

HUD Real Time

```
import socket
import json
import threading
import cv2
import time
import os
from IPython.display import display, clear_output
from PIL import Image
```

Maps internal instruction codes (keys) → human-readable messages (values).

```
INSTRUCTION_MAP = {
    "go": "GO",
    "goForward": "GO FORWARD",
    "goLeft": "GO LEFT",
    "stop": "STOP",
    "stopLeft": "STOP LEFT",
    "warning": "WARNING",
    "warningLeft": "WARNING LEFT",
}
```

This code launches a patched DeepStream inference pipeline (with display disabled) and runs a parallel UDP listener to receive detection results in JSON format. It uses multithreading and a shared, thread-safe state to continuously update the latest detections. The system operates in near real-time, with minor latency due to model inference and communication overhead, and serves as a bridge between perception and downstream applications such as AR overlay.

```
import socket, threading, subprocess, json, time
# Stop any existing listener
try:
```

```
    _stop_listener.set()
    time.sleep(2)
    print("Old listener stopped")
except:
    pass

# Correct paths
ORIG_PIPELINE = "/home/logicpro09/omniview_ai/omniview_pipeline.py"
PATCHED_PIPELINE = "/home/logicpro09/omniview_ai/omniview_pipeline_patched.py"
BASE_DIR = "/home/logicpro09/omniview_ai"
VIDEO_FIX = "/home/logicpro09/omniview_ai/output_ds_3_reenc.mp4"

# ========================
# Create patched pipeline
# ========================
with open(ORIG_PIPELINE, "r") as f:
    code = f.read()

# Patch display off for notebook
code = code.replace("DISPLAY      = True", "DISPLAY      = False")
code = code.replace("DISPLAY = True", "DISPLAY = False")

# Keep .pt model paths as is
# Patch video path
code = code.replace('VIDEO_PATH   = "/home/logicpro09/omniview_ai/output_ds_3_reenc.mp4"',
                    f'VIDEO_PATH   = "{VIDEO_FIX}"')

# Prevent crash when no frames are processed
code = code.replace(
    '"min_infer_ms":           np.min(infer_times),',
    '"min_infer_ms":           float(np.min(infer_times)) if len(infer_times) else 0.0,'
)
code = code.replace(
    '"max_infer_ms":           np.max(infer_times),',
    '"max_infer_ms":           float(np.max(infer_times)) if len(infer_times) else 0.0,'
)
code = code.replace(
    '"avg_infer_ms":           np.mean(infer_times),',
    '"avg_infer_ms":           float(np.mean(infer_times)) if len(infer_times) else 0.0,'
)
```

```
with open(PATCHED_PIPELINE, "w") as f:
    f.write(code)

print("Patched pipeline created successfully")

# ==========================
# Shared state
# ==========================
latest_payload = {"sequence": -1, "detections": []}
payload_lock = threading.Lock()
_stop_listener = threading.Event()

# ==========================
# Start pipeline
# ==========================
def run_pipeline():
    process = subprocess.Popen(
        ["python3", PATCHED_PIPELINE],
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        text=True,
        bufsize=1
    )
    for line in process.stdout:
        print(f"[pipeline] {line}", end="")

pipeline_thread = threading.Thread(target=run_pipeline, daemon=True)
pipeline_thread.start()
print("Pipeline started, waiting 25s for models to load...")
time.sleep(25)
```

UPD listener

```
# Start listener after pipeline is running
def udp_listener():
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind(("0.0.0.0", 5055))
```

```
        sock.settimeout(1.0)
        print("Listener bound on port 5055")
        while not _stop_listener.is_set():
            try:
                data, _ = sock.recvfrom(65535)
                parsed = json.loads(data.decode("utf-8"))
                with payload_lock:
                    latest_payload.update(parsed)
            except socket.timeout:
                continue
            except Exception as e:
                print(f"Listener error: {e}")
                break
        sock.close()

    _stop_listener.clear()
    threading.Thread(target=udp_listener, daemon=True).start()
    print("Listener started, waiting 10s for data...")
    time.sleep(10)

    # Check
    with payload_lock:
        p = latest_payload.copy()

    print(f"Sequence  : {p.get('sequence', '?')}")
    print(f"Detections: {len(p.get('detections', []))}")
    if p.get('detections'):
        print(f"Sample: {p['detections'][0]}")
    else:
        print("Still empty - check [pipeline] lines above")
```

OmniView AI - UDP Live HUD

This notebook renders a live HUD overlay on video frames using detection results received through UDP.
It combines traffic-sign instructions, blind-spot alerts, object boxes, and simple velocity estimation.

```
    # ============================================================
    # OMNIVIEW AI - UDP LIVE HUD
```

```
# ==========================================================

import cv2
import time
import os
import numpy as np
import imageio.v2 as imageio
from IPython.display import display, clear_output, Image as IPImage
from PIL import Image as PILImage

VIDEO_PATH  = "/home/logicpro09/omniview_ai/output_ds_3_reenc.mp4"
OUTPUT_PATH = "/home/logicpro09/omniview_hud_udp_final.mp4"
GIF_PATH    = "/home/logicpro09/omniview_hud_udp_final.gif"

MAX_FRAMES = 120
GIF_FRAMES = 40
GIF_FPS = 10

INSTRUCTION_MAP = {
    "go": "GO",
    "goForward": "GO FORWARD",
    "goLeft": "GO LEFT",
    "stop": "STOP",
    "stopLeft": "STOP LEFT",
    "warning": "WARNING",
    "warningLeft": "WARNING LEFT",
}
```

Velocity Tracker

This section defines a simple motion tracker that estimates object movement between frames using center position and bounding-box area.
It generates motion labels such as STATIC, APPROACH, RECEDING, or MERGING.

```
# ==========================================================
# VELOCITY TRACKER
# ==========================================================
```

```
class VelocityTracker:
    def __init__(self):
        self.history = {}

    def update(self, obj_key, cx, cy, area, frame_idx):
        prev = self.history.get(obj_key)
        velocity = None

        if prev is not None:
            frames_elapsed = max(frame_idx - prev["frame"], 1)

            dx = (cx - prev["cx"]) / frames_elapsed
            dy = (cy - prev["cy"]) / frames_elapsed
            speed_px = np.sqrt(dx**2 + dy**2)
            area_change = (area - prev["area"]) / max(prev["area"], 1)

            velocity = {
                "dx": dx,
                "dy": dy,
                "speed_px": speed_px,
                "area_change": area_change,
            }

        self.history[obj_key] = {
            "cx": cx,
            "cy": cy,
            "area": area,
            "frame": frame_idx,
        }
        return velocity

    def get_label(self, velocity):
        if velocity is None:
            return ""

        area_change = velocity["area_change"]
        speed_px = velocity["speed_px"]
        dx = velocity["dx"]

        if speed_px < 1.5:
            return "STATIC"
```

```
        if area_change > 0.08:
            direction = "APPROACH"
        elif area_change < -0.08:
            direction = "RECEDING"
        else:
            if abs(dx) > 3:
                direction = "MERGING-L" if dx < 0 else "MERGING-R"
            else:
                direction = "MOVING"

        return f"{direction} {speed_px:.0f}"

    def get_fast_approach(self, velocity):
        if velocity is None:
            return False
        return velocity["area_change"] > 0.15 and velocity["speed_px"] > 3
```

Open Input Video and Initialize Output

This cell opens the source video, reads frame properties, prepares the output video writer, defines blind-spot regions, and initializes the tracker and counters.

```
# ============================================================
# OPEN VIDEO
# ============================================================
cap = cv2.VideoCapture(VIDEO_PATH)
if not cap.isOpened():
    raise ValueError(f"Could not open video: {VIDEO_PATH}")

fps = cap.get(cv2.CAP_PROP_FPS)
if fps <= 0:
    fps = 30.0

w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

out = cv2.VideoWriter(
```

```
        OUTPUT_PATH,
        cv2.VideoWriter_fourcc(*"mp4v"),
        fps,
        (w, h)
    )

    print(f"Video: {w}x{h} @ {fps:.1f}fps")

    left_zone = (0, int(h * 0.48), int(w * 0.22), h)
    right_zone = (int(w * 0.78), int(h * 0.48), w, h)

    tracker = VelocityTracker()
    frame_idx = 0
    total_detections = 0

    print("Starting HUD loop...")
```

Main HUD Processing Loop

This cell reads each frame, receives the latest UDP detection payload, draws the HUD layout, processes detections, adds alerts and instruction banners, and writes the output video.

```
    # ============================================================
    # MAIN LOOP
    # ============================================================
    while frame_idx < MAX_FRAMES:
        ok, frame = cap.read()
        if not ok:
            break

        with payload_lock:
            payload = latest_payload.copy()

        detections = payload.get("detections", [])
        left_alert = False
        right_alert = False
        instruction_text = None
        instruction_conf = 0.0
```

```
        frame_dets = 0

        # -------------------------
        # Top bar
        # -------------------------
        cv2.rectangle(frame, (0, 0), (w, 60), (20, 20, 20), -1)
        cv2.putText(
            frame,
            "OmniView AI | UDP Live HUD",
            (20, 38),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.9,
            (255, 255, 255),
            2
        )

        # -------------------------
        # Blind-spot overlays
        # -------------------------
        overlay = frame.copy()
        cv2.rectangle(overlay, (left_zone[0], left_zone[1]), (left_zone[2], left_zone[3]), (0, 255, 255), -1)
        cv2.rectangle(overlay, (right_zone[0], right_zone[1]), (right_zone[2], right_zone[3]), (0, 255, 255), -1)
        cv2.addWeighted(overlay, 0.18, frame, 0.82, 0, frame)

        cv2.rectangle(frame, (left_zone[0], left_zone[1]), (left_zone[2], left_zone[3]), (0, 255, 255), 2)
        cv2.rectangle(frame, (right_zone[0], right_zone[1]), (right_zone[2], right_zone[3]), (0, 255, 255), 2)

        cv2.putText(
            frame,
            "LEFT BLIND SPOT",
            (10, left_zone[1] - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.55,
            (0, 255, 255),
            2
        )
        cv2.putText(
            frame,
            "RIGHT BLIND SPOT",
            (right_zone[0], right_zone[1] - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
```

```
        0.55,
        (0, 255, 255),
        2
    )

    # -------------------------
    # Process detections
    # -------------------------
    for det in detections:
        conf = float(det.get("confidence", 0.0))
        label = str(det.get("label", "object")).strip()
        model_name = str(det.get("model", "")).strip().lower()
        blind_spot = str(det.get("blind_spot", "none")).strip().lower()
        label_key = label.replace(" ", "")

        if model_name == "lisa" and conf < 0.15:
            continue
        if model_name == "coco" and conf < 0.25:
            continue
        if model_name not in {"lisa", "coco"}:
            continue

        if model_name == "lisa" and label_key in INSTRUCTION_MAP:
            if conf > instruction_conf:
                instruction_conf = conf
                instruction_text = INSTRUCTION_MAP[label_key]

        xc = float(det.get("x_center", 0.0)) * w
        yc = float(det.get("y_center", 0.0)) * h
        bw = float(det.get("width", 0.0)) * w
        bh = float(det.get("height", 0.0)) * h

        x1 = max(0, min(w - 1, int(xc - bw / 2)))
        y1 = max(0, min(h - 1, int(yc - bh / 2)))
        x2 = max(0, min(w - 1, int(xc + bw / 2)))
        y2 = max(0, min(h - 1, int(yc + bh / 2)))

        if x2 <= x1 or y2 <= y1:
            continue

        cx = int(xc)
```

```
            cy = int(yc)
            area = max((x2 - x1) * (y2 - y1), 1)

            vel_label = ""
            fast_approach = False

            if model_name == "coco":
                obj_key = f"{label}_{int(cx / max(w,1) * 8)}_{int(cy / max(h,1) * 6)}"
                velocity = tracker.update(obj_key, cx, cy, area, frame_idx)
                vel_label = tracker.get_label(velocity)
                fast_approach = tracker.get_fast_approach(velocity)

            color = (0, 255, 0)

            if model_name == "coco":
                if blind_spot == "left":
                    color = (0, 0, 200) if not fast_approach else (0, 0, 255)
                    left_alert = True
                elif blind_spot == "right":
                    color = (0, 0, 200) if not fast_approach else (0, 0, 255)
                    right_alert = True
                elif fast_approach:
                    color = (0, 165, 255)

            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
            cv2.circle(frame, (cx, cy), 4, color, -1)

            short_label = label[:8]
            text = f"{short_label} {conf:.2f}"

            if model_name == "coco" and blind_spot != "none":
                text += f" | {blind_spot.upper()}"

            if vel_label:
                text += f" | {vel_label}"

            text_y = y1 - 5 if y1 > 40 else y1 + 25

            cv2.putText(
                frame,
                text,
```

```
            (x1, text_y),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.45,
            color,
            2
        )

        frame_dets += 1

    total_detections += frame_dets

    # -------------------------
    # Traffic sign banner
    # -------------------------
    if instruction_text:
        cv2.rectangle(frame, (w - 340, 10), (w - 20, 50), (255, 140, 0), -1)
        cv2.putText(
            frame,
            f"SIGN: {instruction_text}",
            (w - 325, 38),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.72,
            (255, 255, 255),
            2
        )

    # -------------------------
    # Alert banners
    # -------------------------
    if left_alert:
        cv2.rectangle(frame, (20, 75), (460, 115), (0, 0, 200), -1)
        cv2.putText(
            frame,
            "ALERT: LEFT BLIND SPOT",
            (35, 103),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.8,
            (255, 255, 255),
            2
        )
```

```
        if right_alert:
            cv2.rectangle(frame, (20, 125), (470, 165), (0, 0, 200), -1)
            cv2.putText(
                frame,
                "ALERT: RIGHT BLIND SPOT",
                (35, 153),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (255, 255, 255),
                2
            )

        # -------------------------
        # Debug footer
        # -------------------------
        cv2.putText(
            frame,
            f"Frame:{frame_idx}  Dets:{frame_dets}  Seq:{payload.get('sequence', '?')}",
            (20, h - 20),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (255, 255, 255),
            2
        )

        out.write(frame)

        if frame_idx % 5 == 0:
            rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            clear_output(wait=True)
            #uncomment this when running
           # display(PILImage.fromarray(rgb))
            print(
                f"frame={frame_idx}/{MAX_FRAMES} | "
                f"dets={frame_dets} | total={total_detections} | "
                f"seq={payload.get('sequence', '?')}"
            )

        frame_idx += 1
        time.sleep(1.0 / fps)
```

Cleanup

This cell releases video resources and stops the UDP listener after processing is complete.

```
# ============================================================
# CLEANUP
# ============================================================
cap.release()
out.release()
_stop_listener.set()

print(f"\nDone. {frame_idx} frames | {total_detections} total detections")
print(f"Saved: {OUTPUT_PATH} ({os.path.getsize(OUTPUT_PATH)} bytes)")
```

Convert Output Video to GIF

This final step creates a short GIF preview from the generated HUD video for quick visualization inside the notebook or presentation.

```
# ============================================================
# GIF + DISPLAY
# ============================================================
print("Converting to GIF...")

gif_frames = []
reader = imageio.get_reader(OUTPUT_PATH)
for i, f in enumerate(reader):
    if i >= GIF_FRAMES:
        break
    gif_frames.append(f)
reader.close()

imageio.mimsave(GIF_PATH, gif_frames, fps=GIF_FPS)
print(f"GIF: {GIF_PATH} ({len(gif_frames)} frames, {len(gif_frames)/GIF_FPS:.1f}s)")
#uncomment before running
display(IPImage(filename=GIF_PATH))
```